

9. CIRCULAR QUEUE USING ARRAY

SOURCE CODE:

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

int isFull() { return (rear + 1) % MAX == front; }
int isEmpty() { return front == -1; }

void enqueue(int val) {
    if (isFull()) { printf("Queue is Full (Overflow)\n"); return; }
    if (front == -1) front = 0;
    rear = (rear + 1) % MAX;
    queue[rear] = val;
    printf("Inserted: %d\n", val);
}

void dequeue() {
    if (isEmpty()) { printf("Queue is Empty (Underflow)\n"); return; }
    printf("Deleted: %d\n", queue[front]);
    if (front == rear) front = rear = -1;
    else front = (front + 1) % MAX;
}

void display() {
    if (isEmpty()) { printf("Queue is Empty\n"); return; }
    printf("Queue: ");
    int i = front;
    while (1) {
        printf("%d ", queue[i]);
        if (i == rear) break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}

int main() {
    int choice, val;
    while (1) {
        printf("\n1.Enqueue 2.Dequeue 3.Display 4.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: printf("Enter value: "); scanf("%d", &val); enqueue(val); break;
            case 2: dequeue(); break;
            case 3: display(); break;
            case 4: return 0;
            default: printf("Invalid choice\n");
        }
    }
}
```

PROGRAM OUTPUT

Output

Clear

1.Enqueue 2.Dequeue 3.Display 4.Exit

Enter choice: 1

Enter value: 10

Inserted: 10

1.Enqueue 2.Dequeue 3.Display 4.Exit

Enter choice: 1

Enter value: 20

Inserted: 20

1.Enqueue 2.Dequeue 3.Display 4.Exit

Enter choice: 3

Queue: 10 20

1.Enqueue 2.Dequeue 3.Display 4.Exit

Enter choice: 2

Deleted: 10

1.Enqueue 2.Dequeue 3.Display 4.Exit

Enter choice: 4